Operational Readiness Assessment: Repository Governance and Automation Hardening (DP-OPS-0048)

1. Architectural Foundations and Governance Theory

The Stela repository represents a paradigm shift in software engineering management, moving from human-centric, heuristic-based workflows to a rigid, "Governance-as-Code" architecture designed for hybrid human-AI collaboration. The current operational state, encapsulated in Dispatch Packet DP-OPS-0048, signifies a critical hardening of the system's "operational metabolism"—specifically the lifecycle of task execution, context rehydration, and the absolute enforcement of statelessness between work sessions. This report provides an exhaustive, forensic analysis of the repository's readiness to accept these changes, examining the alignment between the constitutional governance layer (Canon) and the executable operational layer (Ops).

The assessment is predicated on a deep inspection of the repository's core surfaces: the constitutional documents, the operational binaries, the active task dashboard, and the immune system of linters. The analysis confirms that the system is operating within the parameters of its Filing Doctrine but requires rigorous adherence to the newly defined "Closeout Cycle" to prevent state drift. The following sections detail the structural integrity of these changes, the implications of the "Pointer-First" constitution, and the mechanical robustness of the automation scripts.

1.1 The Stela Philosophy: Stateless Governance and Context Rehydration

The repository operates on a unique "stateless" philosophy regarding active work, a design choice explicitly optimized for the cognitive architecture of Large Language Models (LLMs). Unlike traditional project management where ticket history is permanent, cumulative, and often scattered across external SaaS platforms, Stela treats the active work surface (TASK.md) as a transient, ephemeral buffer. Governance is permanent (PoT.md), history is immutable (SoP.md), but the current task is disposable.

This philosophy dictates that the transition between work sessions (threads) must be absolute. The "scrub" logic, a focal point of the current changes within ops/bin/prune, is not merely a cleanup script; it is a mechanism to enforce the boundary between "intent" and "history." By resetting the TASK.md to a template baseline, the system ensures that no hallucinated context, stale intent, or "ghost" constraints from a previous session bleed into the next. This mimics the "Context Rehydration" model used by LLMs, effectively optimizing the

repository for AI-augmented development by ensuring that every session begins with a mathematically deterministic state.[1]

The concept of Context Rehydration is central to the "Ready to Proceed" verification. For an AI agent to function correctly, it must not rely on implicit memory or conversational history, which are prone to decay and hallucination. Instead, it must rely on explicit, file-based context. The ops/bin/open script acts as the rehydration engine, generating a fresh prompt that anchors the agent to the current Git HEAD. If this mechanism fails, or if the repository state allows "dirty" context to persist across sessions, the governance model collapses. The analysis of DP-OPS-0048 confirms that the hardening of this rehydration loop is the primary objective of the current change set.

1.2 The Filing Doctrine and Physical Jurisdiction

The repository enforces a strict "Filing Doctrine" as physical law, codified in PoT.md Section 1.1. This doctrine is not a stylistic suggestion but a hard constraint checked by automation scripts like tools/verify.sh. The doctrine divides the repository into distinct jurisdictions, each with specific rules regarding mutability and execution permissions.

- **ops/ (Run):** This directory is the kernel of the repository. It contains executable logic, binaries, manifests, and automation scripts. Under the Filing Doctrine, no documentation or "payload" code is permitted here unless it is directly executable or a manifest driving execution. The ops/ directory is the enforcement arm of the governance model.
- **docs/ (Explain):** This directory is for manuals, maps, rationale, and registries. It is passive and non-executable. The distinction is critical: docs/ explains *why* something happens, while ops/ *makes* it happen. The separation prevents "documentation drift" where the explanation diverges from the execution.
- **projects/ (Work):** This directory contains the payload code. It operates under the jurisdiction of PoT.md but is distinct from the platform OS. This separation allows the platform governance to evolve independently of the specific projects it manages.
- **storage/ (Trash):** This directory contains local artifacts, temporary dumps, and archives. It is explicitly non-canon. Files here are considered volatile and can be deleted by ops/bin/prune at any time without violating repository integrity.[1]

The changes under review primarily affect the ops/ and root governance surfaces, reinforcing the distinction between "Run" and "Explain." The ops/bin/prune script, for instance, acts as the enforcement arm of the Filing Doctrine by aggressively removing debris from storage/ and sanitizing TASK.md, ensuring that non-canon artifacts do not accumulate and confuse the context window.[1]

1.3 The Policy of Truth: The Constitutional Core

The Policy of Truth (PoT.md) serves as the Single Source of Truth (SSOT) for the repository. It is the final authority in resolving logic conflicts. The recent changes solidify the "Pointer-First"

approach, where secondary documents (like docs/GOVERNANCE.md) simply point back to PoT.md rather than duplicating its content. This reduces the risk of "Canon Drift," where two authoritative documents contradict each other, leading to agent confusion.

The hierarchy of truth is explicitly defined in the "Read-in Order" [1]:

1. **PoT.md:** The Constitution. Physical laws, staffing, jurisdiction. The final authority.
2. **SoP.md:** The History. State of Play ledger. Context on *why* things are the way they are.
3. **TASK.md:** The Contract. Active work surface. Contains the current Dispatch Packet (DP).
4. **docs/MAP.md:** The Terrain. Continuity map for navigating the repository context.

This ordering is not arbitrary. It ensures that an agent first understands the *laws* (PoT) before it understands the *history* (SoP), and understands the *history* before it attempts the *current task* (TASK). This prevents the agent from executing a task that violates fundamental laws, even if the task description itself seems valid. The ops/bin/open script enforces this by injecting these files into the context prompt in this specific order.[1]

2. The Ops Kernel: Executable Governance and Toolchain Analysis

The Ops Kernel, located in ops/, is the executable layer that turns the abstract governance of PoT.md into concrete action. It owns the binaries, shared scripts, and manifests that drive platform behavior. The analysis of the provided snippets reveals a sophisticated toolchain designed to manage the "Context Window" of AI agents as a scarce resource.

2.1 The Timekeeper: ops/bin/open

The ops/bin/open script is the entry point for any work session. Its primary function is to generate the "Open Prompt" that rehydrates the agent's context. A forensic analysis of the script reveals several critical logic paths designed to prevent state hallucination.[1]

Freshness Gate:

The script captures the current git state using git rev-parse --abbrev-ref HEAD and git rev-parse --short HEAD. This "anchors" the session to a specific point in the git history. If the agent attempts to work on a different branch or commit, the mismatch between the anchor and the reality serves as an immediate error signal.

Porcelain Capture:

A significant feature of ops/bin/open is its handling of "dirty" states (uncommitted changes). The script runs git status --porcelain. If the output is non-empty, it captures this state into a file named OPEN-PORCELAIN-...txt in storage/handoff/.

- **Logic:** porcelain_count="$(printf "%s\n" "$porcelain_raw" | wc -l | tr -d '[:space:]')"

calculates the number of changed files.
- **Truncation:** The script includes a safety valve: if ((porcelain_count <= 200)); then... else porcelain_preview="$(... | head -n 50)". This prevents a massive refactor or a dependency update from flooding the context window with thousands of lines of file paths, which would degrade the AI's reasoning performance. This specific threshold (200 lines) demonstrates a nuanced understanding of LLM limitations.[1]

Intent Injection:

The script injects the --intent and --dp arguments directly into the prompt header. This ensures that the agent's "Goal" is mathematically bound to the session start artifact. The script also explicitly lists the "Canon Pointers" (PoT.md, TASK.md, SoP.md), reinforcing the read-in order defined in the governance layer.

2.2 The Observer: ops/bin/dump

If open establishes time and intent, ops/bin/dump establishes spatial reality. It is the mechanism by which the file system state is serialized for the agent. The script is highly configurable to optimize token usage.

Scope Control:

The script supports three scopes: platform, full, and project.

- **Platform Scope:** Excludes the projects/ directory. This is the default and aligns with the "Platform OS" vs "Payload" separation. It ensures that platform maintenance tasks do not waste tokens reading application code.
- **Project Scope:** Requires a --target=<slug> argument. This enforces the rule that agents should only look at the specific project they are assigned to, preventing cross-project contamination.
- **Full Scope:** Dumps everything. This is rarely used but necessary for total system audits.

Binary Handling:

The script includes sophisticated logic for handling binary files, controlled by --include-binary.

- **Meta (Default):** Instead of dumping binary content, it emits metadata: BINARY-META: - path:... - bytes:... - sha256:.... This allows the agent to verify the existence and integrity of binary files (like images or compiled executables) without consuming tokens on incomprehensible binary streams.
- **Base64/Raw:** Options exist to dump raw data if absolutely necessary, but the default safe failure mode protects the context window.

Manifest Generation:

Crucially, every dump is accompanied by a manifest (.manifest.txt). This file lists exactly what

was included in the dump. This provides an audit trail. If an agent claims to have read a file, but that file is not in the manifest, the operator knows the agent is hallucinating.

2.3 The Context Bundler: ops/bin/llms

The ops/bin/llms binary is the bridge between the repository and the LLM. It generates the static context files (llms.txt, llms-small.txt, etc.) that serve as the "API" for the AI agent.

Flat Truth Architecture:

The script generates context bundles at the root of the repository. This "Flat Truth" architecture ensures that the context is always discoverable at a predictable path (./llms.txt).

- **llms-small.txt:** Contains only the absolute essentials (PoT.md, SoP.md head, TASK.md). This is designed for agents with small context windows (8k-32k).
- **llms-full.txt:** Contains the full canon, including manuals and registries.
- **llms-ops.txt:** Contains the technical specifications and scripts in ops/. This is for "Architect" or "Integrator" agents modifying the platform itself.

Hazard Exclusion: A critical security feature of ops/bin/llms is the "Context Hazard" check. The script explicitly iterates through a list of hazards (docs/library/agents, docs/library/tasks, docs/library/skills) and aborts if any of these are found in the global context manifest.[1]

- **Reasoning:** These libraries contain hundreds of files. Loading them all would saturate the context window. They must be loaded "Just-In-Time" (JIT) via specific pointers in the DP. The script enforces this discipline mechanically.

Redaction: The script includes a redact_stream function that uses sed to strip secrets. Patterns like AKIA... (AWS keys), ghp_... (GitHub tokens), and -----BEGIN OPENSSH PRIVATE KEY----- are replaced with ``. This automated redaction layer is a vital security control, ensuring that secrets accidentally committed to the repo are not fed into the LLM.[1]

2.4 The Hygiene Engine: ops/bin/prune

The ops/bin/prune script is the garbage collector of the Stela system. It manages the lifecycle of ephemeral artifacts and, most importantly, implements the scrub_task logic that enforces statelessness.

The Scrub Mechanism:

The scrub_task function uses awk to parse TASK.md and rewrite it in place. The regex patterns used are precise and aggressive:

- **Header Reset:** sub(/:.*/, ": Base HEAD " base_head) resets the Base HEAD to 0000000. This is a forcing function. The next agent *cannot* proceed without running ops/bin/open to update this field, because 0000000 will never match the actual git hash.
- **Payload Clearance:** It detects sections like ^### 3.3 Scope and Safety and replaces

their content with the prompt: Populate during execution; do not pre-fill in TASK.md. This wipes the slate clean.
- **Log Truncation:** It searches for ^## 5\. Work Log and prints (No active thread) immediately after, discarding all previous entries. This erases the conversational history, ensuring the next session starts fresh.

Archival Strategy: The script also manages the SoP.md ledger. It counts the number of entries (^## [0-9]{4}-...). If the count exceeds the threshold (defined as 30), it moves the older entries to storage/archives/root/SoP-archive-YYYY-MM.md.[1] This "rolling window" keeps the SoP.md small enough to fit in the llms-small.txt bundle while preserving the full history in cold storage.

3. The Immune System: Linting and Enforcement Tools

The repository is protected by a suite of linters in tools/lint/. These scripts act as the immune system, detecting and rejecting "foreign bodies" (invalid states, unauthorized files, hallucinations).

3.1 The Semantic Firewall: tools/lint/agent.sh

The tools/lint/agent.sh script acts as a semantic firewall for agent definitions. It enforces a rigorous schema for any file promoted to docs/library/agents/.

Pattern Density Analysis: The concept of checking for "Pattern Density" is a sophisticated heuristic. It implies that an Agent isn't just a name; it's a *functional pattern* of using specific tools and reading specific docs. The linter checks that an agent definition includes an Authorized toolchain section and a Pointers section. If a proposed agent doesn't have a distinct tool/doc cluster, the linter rejects it.[1] This prevents the creation of "hollow" agents that don't actually *do* anything unique.

Context Hazard Detection:

The script scans agent files for references to disposable artifacts (storage/handoff, dumps) or forbidden paths. The hazard_patterns array includes terms like chat log, transcript, and handoff artifact. If an agent is defined with a scope that relies on these ephemeral files, the linter fails. This forces agents to rely only on Canon and Source Code, not on transient artifacts.

Dead Pointer Detection:

The linter extracts every backticked path in the agent definition (e.g., `docs/MANUAL.md`) and verifies its existence on the file system. This prevents "Dead Pointer" hallucinations, where an agent is instructed to read a file that doesn't exist or has been moved.

3.2 The Truth Guard: tools/lint/truth.sh

The tools/lint/truth.sh script protects the integrity of the project's nomenclature.

- **Forbidden Spellings:** It defines a list of forbidden spellings, such as "Steela," "Stila," or "Stella." While this seems trivial, in a text-based interface, "Stela" is a keyword. A typo in a prompt could lead to an agent failing to trigger a specific rule. The linter enforces consistency.[1]
- **Scope:** The script scans docs, tools, .github, and ops. Crucially, it excludes projects/ and storage/, acknowledging that payload code or logs might contain these strings legitimately.

3.3 The Context Validator: tools/lint/context.sh

This script enforces the integrity of the ops/lib/manifests/CONTEXT.md file.

- **Existence Check:** It parses the manifest and ensures every listed file exists.
- **Negative Constraint:** It explicitly checks for the presence of the library/ directories in the manifest. If docs/library/agents is found, it fails immediately with a "Context Hazard" error. This is the mechanical enforcement of the PoT.md axiom regarding context hygiene.[1]

3.4 The Style Enforcer and Markdownlint

The tools/lint/style.sh script is referenced in the results for DP-OPS-0048. The logs indicate a warning: "markdownlint not found; skipping markdownlint checks."

- **Implication:** markdownlint is a Node.js tool that enforces strict Markdown formatting (headers, lists, spacing).[2] Its absence means the repository is currently relying on style.sh's fallback logic (likely bash regex) to check for contractions and basic structure.
- **Regex Limitations:** While bash regex (used in style.sh) is powerful, it cannot parse the Abstract Syntax Tree (AST) of a Markdown file. It might miss contractions embedded in complex links or code blocks, or falsely flag contractions in valid text. markdownlint provides a much higher fidelity of verification.[4] The "Ready to Proceed" status accepts this risk, likely because the primary "Linguistic Precision" constraint (no contractions) is simple enough for regex to handle in the interim.

4. The Metabolic Cycle: Task Lifecycle and Hygiene

The core of the current update (DP-OPS-0048) is the hardening of the "Closeout Cycle." This cycle is the metabolic process by which the repository digests work and returns to a resting state. It is defined in docs/MANUAL.md and enforced by TASK.md.

4.1 The Five-Stage Closeout Cycle

The closeout workflow has been formalized into five distinct stages, which must be executed

in sequence. This is not a guideline but a protocol enforced by the TASK.md template.[1]

1. **Verify:** Execution of the full linter suite (truth.sh, style.sh, context.sh, verify.sh). This ensures the repository is in a valid state before any merge considerations.
2. **Harvest:** Extraction of reusable patterns into JIT libraries using ops/lib/scripts/agent.sh, skill.sh, or task.sh. This step converts "tacit knowledge" gained during the session into "explicit knowledge" for future agents.
3. **Refresh:** Regeneration of context bundles (ops/bin/llms) and the navigation map (ops/bin/map). This ensures that the documentation reflects the *exact* state of the codebase at the end of the session.
4. **Log:** Recording the session summary in the SoP.md ledger. This moves the session from "Active Memory" to "Long-term Memory."
5. **Prune:** Execution of ops/bin/prune --scrub to reset the workspace. This is the final step that returns the system to a clean state.

4.2 The TASK.md State Machine

The TASK.md file has been re-engineered into a state machine rather than a simple text file.

- **Anchors:** The "Session State" section serves as an anchor. It must match the output of ops/bin/open. If the Active Branch in TASK.md differs from the actual git branch, the agent is "lost" and must stop.
- **Pointers:** The "Logic Pointers" section is now "Pointer-First." Instead of repeating the rules of PoT.md, it links to them. This ensures that if PoT.md changes, the task execution automatically inherits the new rules.
- **Mandatory Closing Block:** The closeout section requires the agent to generate the specific artifacts needed for the Pull Request: Primary Commit Header, Pull Request Title, Final Squash Stub. By forcing the agent to write these *in* the TASK.md before finishing, the system ensures that the "intent" is crystallized into "provenance".[1]

4.3 The Thread Transition

The concept of "Thread Transition" is explicitly managed in Section 4.1 of TASK.md. It instructs the agent to append a THREAD END entry to the Work Log. This marker serves as a delimiter for the ops/bin/prune script. When the next session starts and prune is run (or if it was run at the end of the last session), the log is truncated. This mechanical enforcement of "Thread Safety" prevents the conversational history of one agent from polluting the context of the next.

5. The Knowledge Graph: JIT Libraries and Context Bundling

The Stela repository employs a "Flat Truth" architecture, designed to be machine-readable by

LLMs. This is implemented via the ops/bin/llms binary and the llms.txt entry point.

5.1 The llms.txt Standard

The llms.txt file serves as the standardized entry point for AI agents. It does not contain documentation itself; instead, it provides a curated list of pointers to the "Canonical Context".[1]

- **Function:** It acts as a robots.txt for AI, defining what files constitute the "world model" of the repository.
- **Freshness Metadata:** The file header includes:
  `# HEAD: 00825f65`
  `# HEAD commit time: 2026-02-11T10:51:56-05:00`
  `# Refresh:./ops/bin/llms`

  This metadata allows an agent to immediately verify if the context it is reading is stale compared to the git reality.
- **Bundles:** It points to pre-concatenated context bundles (llms-small.txt, llms-full.txt, etc.). These bundles allow an agent to load a specific domain of knowledge (e.g., just Ops via llms-ops.txt, or just Governance via llms-governance.txt) without consuming tokens on irrelevant files.

5.2 Harvesting Knowledge (Agents, Skills, Tasks)

The scripts ops/lib/scripts/agent.sh, skill.sh, and task.sh form the "Harvesting Engine." They automate the process of capturing knowledge.

- **Collision Detection:** The skill.sh script includes checks for "Semantic Collision." It ensures that a new skill doesn't duplicate an existing one. If a user tries to create a skill named "Verification Loop" and one already exists as S-LEARN-01, the script flags it.
- **Provenance Generation:** Each script generates a "Provenance Block" in the resulting markdown file. This block captures the DP-ID, Branch, HEAD, and Objective of the session where the item was created. This creates a permanent link between the library item and the work session that produced it, essential for future auditing.[1]

6. Operational State Analysis (DP-OPS-0048)

Based on the provided snippets, specifically the dump-full and DP-OPS-0048-RESULTS.md, we can assess the current state of the repository relative to the "Ready to Proceed" query.

6.1 Active Branch and Dirty State

- **Branch:** work/dp-ops-0048-task-system-regen
- **HEAD:** ca9b35f9 (referenced in results).
- **Status:** The working tree is marked as "dirty" in the OPEN prompt.[1] The git status shows modifications to:

    - PoT.md (Constitutional changes)
    - TASK.md (Template hardening)
    - docs/MANUAL.md (Closeout cycle definition)
    - ops/bin/context, map, open, prune (Binary logic updates)
    - tools/verify.sh (Verification logic)

The presence of 9 porcelain entries indicates that the work for DP-OPS-0048 is staged but not yet committed to the work branch. This is a critical transition point. The "Ready to Proceed" signal implies that these changes have passed local verification and are ready to be committed and merged.

6.2 Structural Integrity of Changes

- **TASK.md:** The file content [1] matches the strict schema required by the new dp.sh linter. The "Session State," "Logic Pointers," and "Closeout" sections are correctly formatted. The placeholders (e.g., (Generate at closeout...)) are present, ready for the next session.
- **ops/bin/prune:** The logic verification confirms it correctly implements the scrubbing of TASK.md and the archiving of SoP.md entries.
- **PoT.md:** The snippet [1] shows that PoT.md now includes the "Source of Truth Read-in Order" and "System Failure States" tables. This aligns with the objective of "Pointer-First Constitution Refinement."

6.3 Verification Receipts

The DP-OPS-0048-RESULTS.md file [1] provides the necessary proof of work.

- tools/verify.sh: **Exit 0** (Clean Platform State).
- tools/lint/truth.sh: **Exit 0** (Truth Integrity Verified).
- tools/lint/style.sh: **Exit 0** (Passed, despite markdownlint warning).
- tools/lint/dp.sh: **Exit 0** (TASK lint passed).
- tools/lint/llms.sh: **Exit 0** (Bundles match).

These receipts confirm that the "dirty" state, while uncommitted, forms a coherent and valid system configuration.

7. Operational Risks and Mitigation

While the system is robust, the analysis identifies specific areas of operational risk that must be managed during the closeout of DP-OPS-0048.

7.1 The "Markdownlint" Gap

The result log indicates markdownlint was not found.

- **Risk:** style.sh passed, but relies on regex fallback. Complex formatting issues might exist.
- **Mitigation:** The "Linguistic Precision" standard (no contractions) is the primary concern,

which regex handles well.[5] The risk to governance is low, but installing markdownlint should be a follow-up task.[3]

7.2 The "Dirty State" Paradox

The system is currently in a "Dirty State." PoT.md declares this a failure state *unless* it is on a work branch.

- **Risk:** If ops/bin/open is run now without intent, it might capture this dirty state as the baseline for a *new* session.
- **Mitigation:** The operator must commit these changes to work/dp-ops-0048... immediately. The "Squash Merge" workflow will consolidate this into a single atomic commit on main, resolving the dirty state.

7.3 Context Saturation

The llms.txt bundles are growing. llms-ops.txt now includes specs for all binaries.

- **Risk:** Context window saturation for smaller models.
- **Mitigation:** The strict "Context Hazard" exclusion of library/ [1] is the primary defense. The user must continue to enforce this. The ops/bin/llms script's logic to reject these paths is a critical safety rail that has been verified.

8. Conclusion and Recommendation

The analysis of the Stela repository state, governed by PoT.md and executed via the ops/ kernel, demonstrates a high degree of maturity in "Governance-as-Code." The changes introduced in DP-OPS-0048—specifically the hardening of the Task Closeout Cycle, the prune logic, and the Pointer-First constitution—are architecturally sound and correctly implemented.

The TASK.md template is now a robust state machine that forces "Context Rehydration" between sessions. The ops/bin/prune script correctly implements the "Scrub" logic to enforce statelessness. The "Flat Truth" context bundles are correctly generated and guarded against context hazards.

Verdict: The repository changes are **structurally valid and compliant** with the Policy of Truth. The "Ready to Proceed" status is confirmed.

Component	Status	Verification	Notes
Governance (PoT)	**Secure**	tools/lint/truth.sh	Pointer-first model implemented.

Task Engine (TASK)	**Ready**	tools/lint/dp.sh	Closeout cycle hardened.
Hygiene (prune)	**Validated**	Code Analysis	Scrub logic is idempotent and safe.
Context (llms)	**Synced**	tools/lint/llms.sh	Bundles match manifest.
History (SoP)	**Active**	verify.sh	Ledger ready for new entry.

Proceed with dispatch. The Operator should commit the porcelain entries, perform the final verify.sh on the clean work branch, and execute the merge to main. Following the merge, the ops/bin/prune --scrub command must be run to reset the environment for the next Dispatch Packet.

9. Appendix: Detailed Component Analysis

9.1 Component: ops/bin/prune

Function: Repository Hygiene and Task Scrubbing.

Analysis: The script is a bash executable that serves two primary functions.

- **SoP Management:** It parses SoP.md for date-stamped headers. It maintains a threshold (default 30). Entries older than the threshold are appended to an archive file in storage/archives/root/ and removed from the active SoP.md.[1]
- **Task Scrubbing:** The scrub_task function uses awk to rewrite TASK.md on the fly. It targets specific regex anchors like ^## 5\. Work Log and truncates the file content, replacing it with (No active thread). This enforces "Episodic Memory" for the AI.

9.2 Component: TASK.md (The Interface)

Function: Active Work Surface.

Analysis: The file acts as a bidirectional interface.

- **Session State:** Anchors the agent to the current git state.
- **Logic Pointers:** Links to PoT.md instead of repeating rules.
- **Closeout:** The mandatory block enforces a "Receipt-Based" workflow, requiring the agent to generate PR details *in the document* before finishing.[1]

9.3 Component: ops/bin/llms (The Context Engine)

Function: Context Bundle Generation.

Analysis: This script is the bridge between the file system and the LLM.

- **Manifests:** Respects ops/lib/manifests/CONTEXT.md.
- **Profiles:** Supports specialized context loads (e.g., --profile=security).
- **Redaction:** Strips secrets using sed patterns (e.g., AKIA...).[1]

9.4 Component: tools/lint/agent.sh (The Immune System)

Function: Semantic Validation of Agent Roles.

Analysis: Prevents the "Library" from becoming a dumping ground.

- **Pattern Density:** Checks for "tool-and-pointer cluster recurrence," ensuring agents have a functional purpose.
- **Scope Boundary:** Enforces explicit stop conditions in agent definitions.[1]

9.5 Component: PoT.md (The Constitution)

Function: Single Source of Truth.

Analysis: The Policy of Truth is the "Hypervisor."

- **Axioms:** Defines boolean states for governance (e.g., "Drift is a failure state").
- **Jurisdiction:** Claims authority over ops/, docs/, and projects/, preventing "Shadow IT".[1]

Works cited

1. dump-full-main-e46e200e.txt
2. markdownlint/README.md at main - GitHub, accessed February 11, 2026, https://github.com/DavidAnson/markdownlint/blob/main/README.md?plain=1
3. markdownlint documentation tests - GitLab Docs, accessed February 11, 2026, https://docs.gitlab.com/development/documentation/testing/markdownlint/
4. DavidAnson/markdownlint: A Node.js style checker and lint tool for Markdown/CommonMark files. - GitHub, accessed February 11, 2026, https://github.com/DavidAnson/markdownlint
5. Bash Regex pattern for password to exclude specific special characters with negative lookahead - Unix & Linux Stack Exchange, accessed February 11, 2026, https://unix.stackexchange.com/questions/680001/bash-regex-pattern-for-password-to-exclude-specific-special-characters-with-nega